The following listing provides the simplified code view of the `<ChatList/>` component that implements the functionality I just described (ChatList.jsx).

Listing 2.2 Component that lists all conversation messages

```
import ChatMessage from "./ChatMessage";
import ChatBubbleLoading from "./ChatBubbleLoading";

const ChatList = ({ messages, isLoading }) => {
  return (
    <ul>
      {messages.map((message) => (
        <li key={message?.id}>
          <ChatMessage
            role={message.role}
            text={message.content}
          />
        </li>
      ))}
      {isLoading ? (
        <li>
          <ChatBubbleLoading />
        </li>
      ) : null}
    </ul>
  );
};

export default ChatList;
```

Renders each chat message dynamically using ChatMessage

Shows a loading bubble while the assistant responds

`<ChatList/>` maps over the `messages` array to display each message with `<ChatMessage/>`, ensuring new messages appear in real time with the correct role. When `isLoading` is true, `<ChatBubbleLoading />` provides feedback that the assistant is processing a response.

Now that we've seen how the UI is rendered, let's focus on the engine powering chat interaction: the `useChatFormSubmit` hook. This custom hook is a prime example of isolating and reusing complex logic. Instead of cluttering our `<ChatPage/>` component with state management and API calls, we've encapsulated all of that functionality here. This follows the principle of separation of concerns, which is crucial for building maintainable applications.

> **NOTE** Custom React hooks are reusable functions that elevate React components. They can access and manipulate states (`useState`, `useReducer`) and perform side effects (data fetching, subscriptions) using lifecycle methods (`useEffect`). This enables cleaner component logic and improved state management. Hooks promote code reusability, improve readability by separating complex logic, and enhance testability by isolating functionality.

To provide this functionality, we follow some practical steps for creating a custom hook in React: